DATABASES

database-design-implementation

Table of Contents

Part 1: Revised ERD and schema 2

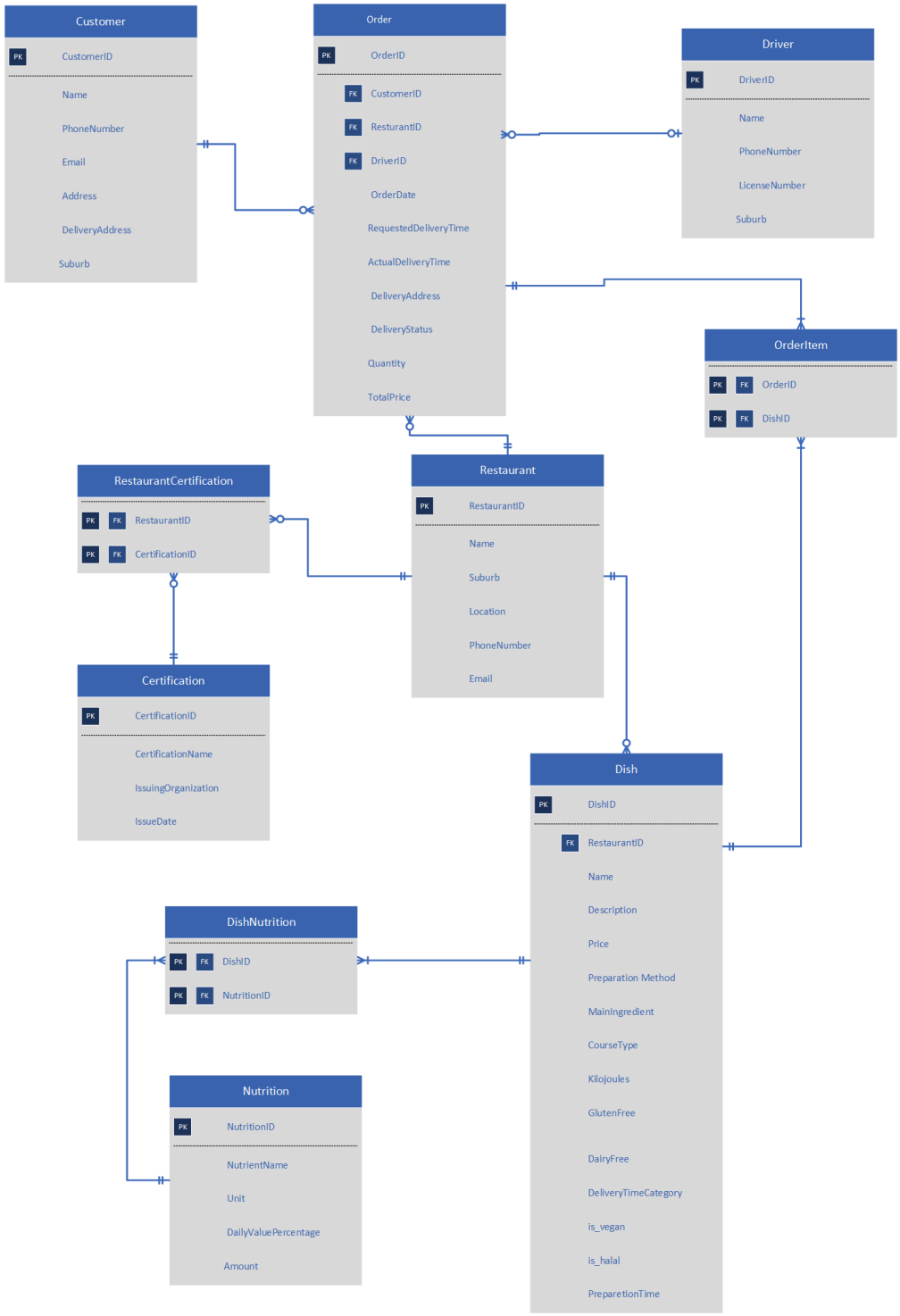
Part 2: Data dictionary 5

Part 3: Implementation 15

Part 4: Views 17

Part 1: Revised ERD and schema

a)



Legends For ERD Diagram:

Symbol	Meaning
	Mandatory—One
	Mandatory—Many
	Optional—One
	Optional—Many

- **Primary Key (PK)**: Represents a unique identifier for each record in a table, ensuring data integrity and quick access.
- **Foreign Key (FK)**: Creates a relationship between tables by linking a column in one table to a primary key in another.
- **Mandatory-One**: Indicates a required, single relationship between entities, where each instance in one entity must have a counterpart in another.
- **Mandatory-Many**: Specifies a required relationship where one entity instance must relate to multiple instances in another.
- **Optional-One**: Shows a non-required, single relationship, where each instance in one entity may or may not relate to one instance in another.
- **Optional-Many**: Indicates a non-required relationship where one instance in one entity can relate to multiple instances in another.

b) Explanation of Changes Made to ERD:

Based on the feedback from Assignment 1 and the new requirements outlined in this assignment 2, I made many adjustments to the ERD to better fulfill FineFoods4U's operational demands and transactional scope. In Assignment 1, some relationships and entities were missing or unclear, necessitating the development of new fields to completely represent the data model needs.

- **Associative entities:**

  OrderItem: I used **OrderItem** to resolve the many-to-many relationship between **Order** and **Dish**, allowing several dishes to be included in a single order while also allowing each dish to appear in multiple orders. This design ensures flexibility and scalability in order details.

  RestaurantCertification: I introduced **RestaurantCertification** to manage the many-to-many relationship between **Restaurant** and **Certification**. This enables one restaurant to hold several certifications (such as vegan or halal).

  DishNutrition: This entity was created to handle the many-to-many relationship between **Dish** and **Nutrition**. As a result, each dish can have a variety of nutritional properties, such as calorie count or dietary information, while each nutrition feature can be applied to numerous dishes.

- **Suburb Attribute for Localized Services:**
  To fulfill the requirement that customers, drivers, and restaurants be restricted to specific suburbs, I introduced a Suburb attribute to the **Customer**, **Driver**, and **Restaurant** entities. This feature enables the database to enforce the requirement that orders can only be placed at restaurants in the customer's suburb. Furthermore, by incorporating Suburb in Driver, the database can manage and allocate available drivers by suburb, resulting in a more efficient delivery procedure.

- **Delivery characteristics in Order Entity:**
  To help Bill track delivery efficiency, I added the **RequestedDeliveryTime** and **ActualDeliveryTime** attributes to the **Order** entity. These attributes are critical for tracking and analyzing delivery times and service quality. This change allows FineFoods4U to maintain responsibility and provide a consistent customer experience.

- **Addition of Legends in the ERD Diagram:**
  The legend provides a quick reference to the symbols and notations used, allowing anyone reading the diagram to appropriately grasp the relationships and entities without having to learn the ERD rules from scratch. This extra aspect is consistent with input from Assignment 1, which advised clearer visual explanations. Including legends reflects my attention to detail and dedication to making the ERD as accessible and user-friendly as feasible.

c) Explanation:

1. Each entity has a primary key defined, which uniquely identifies records within that entity.
2. Foreign keys are included to establish relationships between entities, enabling referential integrity across tables.
3. **The schema is in 3NF as:**
   - There are no repeating groups, and each table has atomic attributes.
   - Each non-key attribute depends fully on the primary key, and there are no partial dependencies in tables with composite keys.
   - There are no transitive dependencies, as non-key attributes depend only on the primary key within each table.

Relational Schema:

• **CUSTOMER** (CustomerID, Name, PhoneNumber, Email, Address, DeliveryAddress, Suburb)
• **ORDER** (OrderID, OrderDate, DeliveryAddress, DeliveryStatus, RequestedDeliveryTime, ActualDeliveryTime, Quantity, TotalPrice, **CustomerID**, **RestaurantID**, **DriverID**)
• **DRIVER** (DriverID, Name, PhoneNumber, LicenseNumber, Suburb)
• **RESTAURANT** (RestaurantID, Name, Suburb, Location, PhoneNumber, Email)

- **ORDERITEM** (**OrderID**, **DishID**)
- **DISH** (DishID, Name, Description, Price, PreparationMethod, MainIngredient, CourseType, Kilojoules, GlutenFree, DairyFree, DeliveryTimeCategory, Is_vegan, Is_halal, PreparationTime, **RestaurantID**)
- **RESTAURANTCERTIFICATION** (**RestaurantID**, **CertificationID**)
- **CERTIFICATION** (CertificationID, CertificationName, IssuingOrganization, IssueDate)
- **DISHNUTRITION** (**DishID**, **NutritionID**)
- **NUTRITION** (NutritionID, NutrientName, Unit, DailyValuePercentage, Amount)

Part 2: Data dictionary

a) FineFoods4U Database Data Dictionary:

Table 1: CUSTOMER

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
CustomerID	Unique identifier for each customer	NUMBER	10	Positive integers	None	Yes	Primary Key
Name	Full name of the customer	VARCHAR2	100	Any valid text	None	Yes	NOT NULL
PhoneNumber	Contact phone number	VARCHAR2	15	Valid phone number format	None	Yes	NOT NULL
Email	Customer's email address	VARCHAR2	100	Valid email format	None	Yes	NOT NULL, UNIQUE
Address	Customer's residential address	VARCHAR2	200	Any valid text	None	Yes	NOT NULL
DeliveryAddress	Address for food delivery	VARCHAR2	200	Any valid text	None	Yes	NOT NULL
Suburb	Suburb where customer lives	VARCHAR2	50	Valid suburb name	None	Yes	NOT NULL

Table 2: ORDER

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
OrderID	Unique identifier for each order	NUMBER	10	Positive integers	None	Yes	Primary Key
CustomerID	Reference to customer placing order	NUMBER	10	Valid CustomerID	None	Yes	Foreign Key references CUSTOMER(CustomerID)
RestaurantID	Reference to restaurant fulfilling order	NUMBER	10	Valid RestaurantID	None	Yes	Foreign Key references RESTAURANT(RestaurantID)
DriverID	Reference to assigned delivery driver	NUMBER	10	Valid DriverID	None	Yes	Foreign Key references DRIVER(DriverID)
OrderDate	Date and time order was placed	TIMESTAMP	-	Valid date/time	SYSTIMESTAMP	Yes	NOT NULL
RequestedDeliveryTime	Customer's requested delivery time	TIMESTAMP	-	Valid date/time	None	Yes	NOT NULL
ActualDeliveryTime	Actual time of delivery	TIMESTAMP	-	Valid date/time	None	No	None
DeliveryAddress	Delivery location	VARCHAR2	200	Any valid text	None	Yes	NOT NULL
DeliveryStatus	Current status of the order	VARCHAR2	20	Pending', 'In Progress', 'Delivered', 'Cancelled'	Pending'	Yes	NOT NULL
Quantity	Total number of items in order	NUMBER	5	Positive integers	1	Yes	NOT NULL, CHECK (Quantity > 0)
TotalPrice	Total cost of the order	NUMBER	10,2	Positive decimals	0	Yes	NOT NULL, CHECK (TotalPrice >= 0)

Table 3: DRIVER

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
DriverID	Unique identifier for each driver	NUMBER	10	Positive integers	None	Yes	Primary Key
Name	Full name of driver	VARCHAR2	100	Any valid text	None	Yes	NOT NULL
PhoneNumber	Driver's contact number	VARCHAR2	15	Valid phone number format	None	Yes	NOT NULL
LicenseNumber	Driver's license number	VARCHAR2	20	Valid license format	None	Yes	NOT NULL, UNIQUE
Suburb	Suburb where driver operates	VARCHAR2	50	Valid suburb name	None	Yes	NOT NULL

Table 4: RESTAURANT

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
RestaurantID	Unique identifier for each restaurant	NUMBER	10	Positive integers	None	Yes	Primary Key
Name	Restaurant name	VARCHAR2	100	Any valid text	None	Yes	NOT NULL
Suburb	Restaurant location suburb	VARCHAR2	50	Valid suburb name	None	Yes	NOT NULL
Location	Full address of restaurant	VARCHAR2	200	Any valid text	None	Yes	NOT NULL
PhoneNumber	Restaurant contact number	VARCHAR2	15	Valid phone number format	None	Yes	NOT NULL
Email	Restaurant email address	VARCHAR2	100	Valid email format	None	Yes	NOT NULL

Table 5: ORDERITEM

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
OrderID	Reference to the order	NUMBER	10	Valid OrderID	None	Yes	Primary Key, Foreign Key references ORDER(OrderID)
DishID	Reference to the dish ordered	NUMBER	10	Valid DishID	None	Yes	Primary Key, Foreign Key references DISH(DishID)

Table 6: DISH

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
DishID	Unique identifier for each dish	NUMBER	10	Positive integers	None	Yes	Primary Key
RestaurantID	Reference to restaurant offering dish	NUMBER	10	Valid RestaurantID	None	Yes	Foreign Key references RESTAURANT(RestaurantID)
Name	Name of the dish	VARCHAR2	100	Any valid text	None	Yes	NOT NULL
Description	Detailed description of dish	VARCHAR2	500	Any valid text	None	Yes	NOT NULL
Price	Cost of the dish	NUMBER	8,2	Positive decimals	0	Yes	NOT NULL, CHECK (Price >= 0)
PreparationMethod	How the dish is prepared	VARCHAR2	50	Fried', 'Steamed', 'Baked', 'Raw', etc.	None	Yes	NOT NULL
MainIngredient	Primary ingredient in dish	VARCHAR2	50	Any valid text	None	Yes	NOT NULL
CourseType	Type of course	VARCHAR2	20	Starter', 'Main', 'Dessert', 'Side'	None	Yes	NOT NULL
Kilojoules	Energy content in kilojoules	NUMBER	6	Positive integers	0	Yes	NOT NULL, CHECK (Kilojoules >= 0)

GlutenFree	Indicates if dish is gluten-free	CHAR	1	Y' or 'N'	N'	Yes	NOT NULL, CHECK (GlutenFree IN ('Y','N'))
DairyFree	Indicates if dish is dairy-free	CHAR	1	Y' or 'N'	N'	Yes	NOT NULL, CHECK (DairyFree IN ('Y','N'))
DeliveryTimeCategory	Expected delivery time category	VARCHAR2	20	Fast', 'Regular', 'Worth the wait'	None	Yes	NOT NULL
Is_vegan	Indicates if dish is vegan	CHAR	1	Y' or 'N'	N'	Yes	NOT NULL, CHECK (Is_vegan IN ('Y','N'))
Is_halal	Indicates if dish is halal	CHAR	1	Y' or 'N'	N'	Yes	NOT NULL, CHECK (Is_halal IN ('Y','N'))
PreparationTime	Time needed to prepare dish in minutes	NUMBER	4	Positive integers	None	Yes	NOT NULL, CHECK (PreparationTime > 0)

Table 7: RESTAURANTCERTIFICATION

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
RestaurantID	Reference to certified restaurant	NUMBER	10	Valid RestaurantID	None	Yes	Primary Key, Foreign Key references RESTAURANT(RestaurantID)
CertificationID	Reference to certification type	NUMBER	10	Valid CertificationID	None	Yes	Primary Key, Foreign Key references CERTIFICATION(CertificationID)

Table 8: CERTIFICATION

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
CertificationID	Unique identifier for certification	NUMBER	10	Positive integers	None	Yes	Primary Key
CertificationName	Name of certification	VARCHAR2	50	Any valid text	None	Yes	NOT NULL, UNIQUE
IssuingOrganization	Organization issuing certification	VARCHAR2	100	Any valid text	None	Yes	NOT NULL

IssueDate	Date certification was issued	DATE	-	Valid date	None	Yes	NOT NULL

Table 9: DISHNUTRITION

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
DishID	Reference to the dish	NUMBER	10	Valid DishID	None	Yes	Primary Key, Foreign Key references DISH(DishID)
NutritionID	Reference to nutrition information	NUMBER	10	Valid NutritionID	None	Yes	Primary Key, Foreign Key references NUTRITION(NutritionID)

Table 10: NUTRITION

Column Name	Description	Data Type	Size	Domain	Default	Required	Constraints
NutritionID	Unique identifier for nutrition info	NUMBER	10	Positive integers	None	Yes	Primary Key
NutrientName	Name of the nutrient	VARCHAR2	50	Any valid text	None	Yes	NOT NULL
Unit	Unit of measurement	VARCHAR2	20	g', 'mg', 'mcg', 'IU', etc.	None	Yes	NOT NULL
DailyValuePercentage	Percentage of daily value	NUMBER	5,2	0-100	0	Yes	NOT NULL, CHECK (DailyValuePercentage >= 0 AND DailyValuePercentage <= 100)
Amount	Amount of nutrient	NUMBER	10, 2	Positive decimals	0	Yes	NOT NULL, CHECK (Amount >= 0)

b) Referential Integrity Rules:

Referential integrity rules in database architecture serve to ensure that connected tables are consistent. Specifically, the **ON DELETE** and **ON UPDATE** actions are used to regulate how changes to primary keys affect foreign key references across tables:

ON DELETE: Determines what happens to foreign key records if the referenced main key is

destroyed. For example, a **RESTRICT** action prevents deletion if there are any active references, whereas **CASCADE** deletes all dependant records.

ON UPDATE: Automatically updates any foreign key values when the primary key they reference changes, ensuring that related tables remain synced and data integrity is preserved across relationships.

Below, I outline the referential integrity rules for each foreign key in the FineFoods4U database schema:

1. **ORDER Table:**
   - **CustomerID (references CUSTOMER):**
     ON DELETE: RESTRICT
     ON UPDATE: CASCADE
     Rationale: It is critical that an order maintains the contact with the consumer who placed it. Deleting a customer with ongoing orders should be prohibited, hence a **RESTRICT** on delete is implemented. However, if a client's details (such as name or contact information) are changed, the changes should be reflected in all orders linked with that customer, hence **CASCADE** is used for updates.

   - **RestaurantID (refers to RESTAURANT):**
     ON DELETE: RESTRICT
     ON UPDATE: CASCADE
     Rationale: Orders are linked to certain restaurants. Deleting a restaurant that is presently fulfilling an order should be prohibited because it would result in inconsistent data. On the other hand, if the restaurant's details change (such as its name or address), the corresponding orders should be updated, which is why **CASCADE** on update is used.

   - **DriverID (refers to DRIVER):**
     ON DELETE: RESTRICT
     ON UPDATE: CASCADE
     Rationale: A driver's relationship to an order is critical for delivery management. Deleting a driver associated with an order is not permitted because it would invalidate the delivery record. Any changes to driver information (such as a phone number or assignment area) should cascade to the orders to which the driver is allocated, justifying **CASCADE** updates.

2. **ORDERITEM Table:**
   - **OrderID (refers to ORDER).**
     ON DELETE: CASCADE
     ON UPDATE: CASCADE
     Rationale: To ensure data consistency, when an order is deleted, all associated order items should be removed as well. As a result, **CASCADE** is employed for both deletion and updating operations.

- **DishID (references DISH):**
  ON DELETE: RESTRICT
  ON UPDATE: CASCADE
  Rationale: Deleting a dish from an active order would render the order void. As a result, the ability to delete an active dish is limited. However, adjustments to a dish, such as changes in description or price, should be propagated to associated order items, which is why **CASCADE** is set to update.

3. **DISH Table:**
   - **Restaurant ID (references RESTAURANT)**
     ON DELETE, CASCADE
     ON UPDATE: CASCADE
     Rationale: To preserve uniformity, if a restaurant is deleted, so should all of its dishes. This validates the use of **CASCADE** for both deletion and update operations.

4. **RESTAURANTCERTIFICATION Table:**
   - **Restaurant ID (references RESTAURANT)**
     ON DELETE, CASCADE
     ON UPDATE: CASCADE
     Rationale: Certifications are linked to individual eateries. If a restaurant is deleted, all associated certifications should be removed as well. Thus, **CASCADE** is used for both delete and update operations.

   - **CertificationID (refers to CERTIFICATION)**
     ON DELETE: RESTRICT
     ON UPDATE: CASCADE
     Rationale: A certification cannot be deleted if it is actively used by a restaurant. This ensures that certifications are valid for as long as they are linked to a restaurant. Updates to certification details should cascade to restaurant certification connections, hence **CASCADE** is used for updates.

5. DISHNUTRITION Table:
   - **DishID (references DISH):**
     ON DELETE, CASCADE
     ON UPDATE: CASCADE
     Rationale: When a dish is deleted, the nutrition information should likewise be removed. Thus, **CASCADE** is suitable for both delete and update operations.

   - **NutritionID (References NUTRITION)**
     ON DELETE: RESTRICT
     ON UPDATE: CASCADE
     Rationale: Deleting a nutrition record associated with a dish while it is still in use may result in incomplete data. However, changes to nutrition information should spread, therefore **CASCADE** is employed for updates.

c) **Business Rules (Enterprise Constraints):**

1. **Geographical constraints:**
   - Customers can only order from restaurants in their own suburb, ensuring that delivery logistics are possible and avoiding errors caused by inaccurate addresses.

   - Drivers can only be allocated to deliveries inside their chosen suburb. This maintains delivery management efficiency while reducing operational complexity.

   - Each suburb must have at least one current driver. Every suburb must have drivers available to cover all delivery requirements.

2. **Order Processing**:
   - The total price must be equal to the sum of all order items' prices. This business rule maintains pricing correctness and guarantees that customers are accurately charged.

   - The actual delivery time must come after the requested delivery time. The system must ensure that orders are fulfilled within a reasonable timeframe, avoiding implausible circumstances in which deliveries are marked as taking place before the desired time.

   - The order status must be in the following sequence: Pending → In Progress → Delivered (or cancelled) This ensures that an order's progress is clear and eliminates uncertainty throughout its lifecycle.

   - Orders cannot be placed for delivery times in the past: This guarantees that consumers do not unintentionally or maliciously order food for a previous time.

3. **Restaurant Operations**:
   - Restaurants must offer at least one dish: This ensures that a restaurant will only be included if it has something to offer guests.

   - Restaurants cannot have overlapping certifications. A restaurant cannot be certified for contradictory standards (e.g., halal and non-halal) while maintaining operational consistency.

   - Before accepting orders, restaurants must declare their working hours. This ensures that customers can only place orders during business hours.

4. **Dish Management:**
   - Dish prices must be larger than zero: This guarantees that pricing is correct and prevents errors due to unpriced products.

   - Preparation time must align with the delivery time category: Dish preparation times should be consistent with projected delivery timeframes to ensure customer satisfaction and operational efficiency.

   - Dishes labeled vegan must also be dairy-free to guarantee consistency with vegan consumers' dietary limitations.

   - Halal foods must meet halal certification requirements. This rule ensures the accuracy of dish certificates.

5. **Delivery Management**:
   - A driver's delivery periods cannot overlap: This prevents overbooking and allows for more precise scheduling.

   - Maximum delivery radius within the suburb must be enforced. This ensures logistical efficiency while preventing unrealistic delivery.

   - Estimated delivery times must include dish preparation time. This guarantees that the system generates correct delivery time estimates.

6. **Data Validation**:
   - All phone numbers must be in standard format to ensure data consistency and easy communication.

   - Email addresses should be authentic and unique. This ensures contactability and eliminates duplicate records.

   - Prices and quantities should be positive: This protects the integrity of financial data.

   - Nutritional values must not be negative: This maintains the accuracy and completeness of nutrition data.

7. **Certification Rules**:
   - Certifications must be current and not expired: This assures that restaurants comply with all applicable requirements.

- Restaurants must hold the necessary certificates for specialized cuisine. This rule ensures that specialized dishes satisfy required criteria (e.g., halal).

- Certification renewals must be tracked and enforced. This guarantees that certifications are up to date and avoids expiration issues.

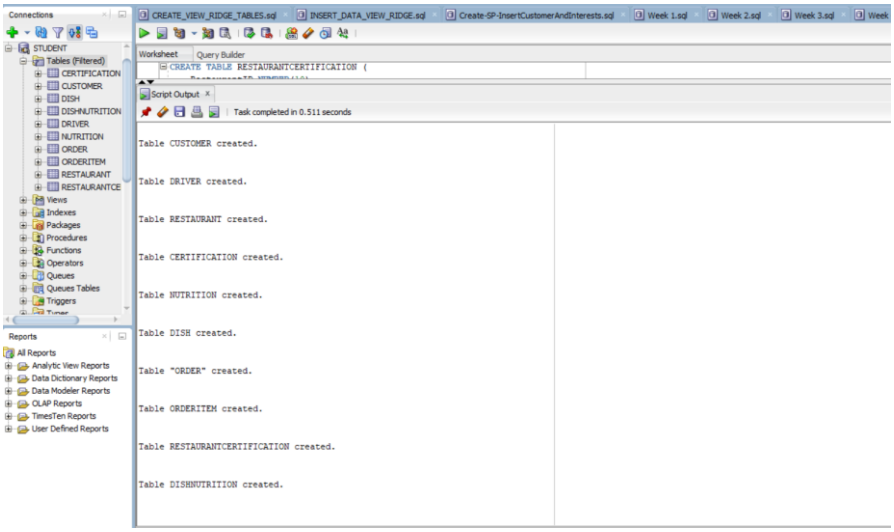
8. **Customer Service**:
   - Customer ratings and feedback must be gathered upon delivery: This allows for ongoing improvement and client satisfaction tracking.

   - Complaints must be reported and resolved within 24 hours. This ensures that client problems are addressed quickly and enhances overall service quality.

   - Customers' delivery addresses must be validated: This guarantees that orders are shipped to accurate and deliverable addresses.

Part 3: Implementation

a) I created each table using my ERD and data dictionary to confirm that the design meets the FineFoods4U case study requirements. Each table relates to an entity, such as a customer, an order, a restaurant, and so on, based on the relationships defined in my ERD. This structure establishes a clear and logical base, encompassing all critical parts of the business concept.
Moreover, I had to be careful while naming the **ORDER** table because Oracle's reserved keywords required me to avoid using only **ORDER** as a table name, Therefore I used (**"ORDER"**) as my table name.



b) To ensure data consistency, I used primary and foreign keys throughout the tables, assigning each a clear and unique identification such as CustomerID, DriverID, and RestaurantID. Foreign key constraints, such as **fk_order_customer** and **fk_order_restaurant**, were used to guarantee referential integrity and maintain relationships between entities. These constraint names are meaningful and easy to understand, indicating which relationships they enforce. This strategy helps to prevent orphaned records by guaranteeing that each Order contains valid customer, restaurant, and driver references, hence strengthening the database's integrity.

```
CONSTRAINT fk_order_customer FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID),
CONSTRAINT fk_order_restaurant FOREIGN KEY (RestaurantID) REFERENCES RESTAURANT(RestaurantID),
CONSTRAINT fk_order_driver FOREIGN KEY (DriverID) REFERENCES DRIVER(DriverID),
CONSTRAINT chk_order_quantity CHECK (Quantity > 0),
CONSTRAINT chk_order_price CHECK (TotalPrice >= 0),
CONSTRAINT chk_delivery_status CHECK (DeliveryStatus IN ('Pending', 'In Progress', 'Delivered', 'Cancelled')),
CONSTRAINT chk_delivery_time CHECK (RequestedDeliveryTime > OrderDate)
```

c) In designing each table, I gave data types and sizes to attributes that best matched the expected data. For example, **NUMBER(10)** is used for ID fields, **VARCHAR2** for text fields such as Name and Email, and **TIMESTAMP** for date fields. By doing so, I was able to assure storage efficiency and enhance query performance while adhering to the data dictionary specifications. The attributes' data types were chosen to accurately reflect genuine values and enable business activities, capturing each feature of the entities with minimal data complexity.

```
CustomerID NUMBER(10) PRIMARY KEY,
Name VARCHAR2(100) NOT NULL,
PhoneNumber VARCHAR2(15) NOT NULL,
Email VARCHAR2(100) NOT NULL UNIQUE,
Address VARCHAR2(200) NOT NULL,
```

d) To ensure consistency between records, I used domain restrictions when appropriate. This contained **CHECK** requirements for data validation, such as the format of phone numbers and emails, permissible values for attributes such as **GlutenFree**, **Is_vegan**, and **DeliveryStatus**, and the requirement that **RequestedDeliveryTime** be later than the purchase date. Using **REGEXP_LIKE** for phone and email formatting improved data validation by ensuring the fields followed patterns. By applying these limitations, I ensured that values remained within acceptable ranges and followed case-specific business rules, resulting in robust and reliable data entry.

```
    CONSTRAINT chk_order_price CHECK (TotalPrice >= 0),
    CONSTRAINT chk_delivery_status CHECK (DeliveryStatus IN ('Pending', 'In Progress', 'Delivered', 'Cancelled')),
    CONSTRAINT chk_delivery_time CHECK (RequestedDeliveryTime > OrderDate)
);
```

e) To make testing easier, I filled the database tables with sample data for all the entities. This data was carefully selected to offer a realistic foundation for testing essential features and performing validation tests.
In addition, I assigned the user MARKERTL the SELECT, UPDATE, and DELETE permissions on all tables, giving the marker complete access to the data as needed. These rights are required for the marking process, allowing the marker to test

functionality, analyse data integrity, and ensure compliance with assignment parameters.

```sql
-- Insert DishNutrition
INSERT INTO DISHNUTRITION VALUES (1, 1);
INSERT INTO DISHNUTRITION VALUES (1, 3);
INSERT INTO DISHNUTRITION VALUES (2, 2);

COMMIT;



-- Part 3: Grant Permissions to Marker
GRANT SELECT, UPDATE, DELETE ON CUSTOMER TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON DRIVER TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON RESTAURANT TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON "ORDER" TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON ORDERITEM TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON DISH TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON RESTAURANTCERTIFICATION TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON CERTIFICATION TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON DISHNUTRITION TO MARKERTL;
GRANT SELECT, UPDATE, DELETE ON NUTRITION TO MARKERTL;
COMMIT;
```

For Question 3, I used the **COMMIT** command after creating and populating the tables to verify that all changes were saved in the database. After granting the necessary permissions to the MARKERTL user, I executed another **COMMIT** command to ensure that the modifications were finalized**.**

Part 4: Views

ViewA:

```sql
CREATE VIEW ViewA AS
SELECT
  o.OrderID,
  o.OrderDate,
  o.RequestedDeliveryTime,
  o.ActualDeliveryTime,
  o.DeliveryAddress,
  o.DeliveryStatus,
  o.Quantity,
  o.TotalPrice,
  r.Name AS RestaurantName,
  d.Name AS DishName,
  d.Description AS DishDescription,
  d.Price AS DishPrice,
  d.PreparationMethod,
  d.MainIngredient,
  d.CourseType,
  d.DeliveryTimeCategory
```

```
FROM "ORDER" o
JOIN ORDERITEM oi ON o.OrderID = oi.OrderID
JOIN DISH d ON oi.DishID = d.DishID
JOIN RESTAURANT r ON d.RestaurantID = r.RestaurantID
WHERE o.CustomerID = 1;
```

Explanation:

This view displays comprehensive details about an order placed by a single customer. It comprises the order ID, date, requested and actual delivery timings, delivery address, and status. In addition, the dishes ordered, and the establishment information are mentioned. This screen allows the driver to follow the dishes and confirm delivery information with the customer. For testing, I assumed **CustomerID = 1**.

ViewB:

```
CREATE VIEW ViewB AS
SELECT
  d.DishID,
  d.Name AS DishName,
  d.Description AS DishDescription,
  d.Price AS DishPrice,
  d.PreparationMethod,
  d.MainIngredient,
  d.CourseType,
  d.DeliveryTimeCategory,
  r.Suburb
FROM DISH d
JOIN RESTAURANT r ON d.RestaurantID = r.RestaurantID
WHERE d.Is_vegan = 'Y'
 AND r.Suburb = 'Mumbai Heights'
 AND d.DeliveryTimeCategory = 'Fast';
```

Explanation:

This view offers a list of vegetarian dishes available for delivery within a specific suburb, with a delivery time of less than 30 minutes. It includes dish titles, descriptions, prices, and the suburb of the restaurant that serves the dish. The criteria **DeliveryTimeCategory = ' Fast'** selects only fast deliveries. For this example, I've assumed that the suburb is called **'Mumbai Heights'.**

ViewC:

```
CREATE VIEW ViewC AS
SELECT
   o.OrderID,
   o.OrderDate,
   o.RequestedDeliveryTime,
   o.ActualDeliveryTime,
   o.DeliveryAddress,
   o.DeliveryStatus,
   o.Quantity,
   o.TotalPrice,
   c.Name AS CustomerName,
   d.Name AS DishName,
   d.Description AS DishDescription,
   d.Price AS DishPrice
FROM "ORDER" o
JOIN ORDERITEM oi ON o.OrderID = oi.OrderID
JOIN DISH d ON oi.DishID = d.DishID
JOIN CUSTOMER c ON o.CustomerID = c.CustomerID
WHERE o.RestaurantID = 1
  AND o.OrderDate = TO_DATE('2024-01-01', 'YYYY-MM-DD');
```

Explanation:
This view retrieves order details for a certain restaurant on a specified date, including customer and dish information. It includes the order ID, client information, and dish data (name, description, and price). I used **RestaurantID = 1** and **OrderDate = '2024-01-01'** as examples to show how this view works.

ViewD:

```
CREATE VIEW ViewD AS
SELECT
   r.Name AS RestaurantName,
   r.Suburb,
   r.Location,
   r.PhoneNumber,
   r.Email,
   d.Name AS DishName,
   d.Description AS DishDescription,
   d.Price AS DishPrice
```

```
FROM RESTAURANT r
JOIN DISH d ON r.RestaurantID = d.RestaurantID
JOIN RESTAURANTCERTIFICATION rc ON r.RestaurantID = rc.RestaurantID
JOIN CERTIFICATION c ON rc.CertificationID = c.CertificationID
WHERE c.CertificationName = 'Vegan Friendly'
  AND d.Is_vegan = 'Y';
```

Explanation:
This view displays vegan-friendly eateries and the dishes they serve. It offers information about vegan dishes such as titles, descriptions, and prices, as well as restaurant specifics (name, suburb, and contact information). Only establishments recognized as "Vegan Friendly" are listed, and only vegan dishes are featured.

ViewE:

```
CREATE VIEW ViewE AS
SELECT
   d.Name AS DriverName,
   d.PhoneNumber AS DriverPhone,
   o.OrderID,
   o.OrderDate,
   c.Name AS CustomerName,
   c.PhoneNumber AS CustomerPhone
FROM DRIVER d
LEFT JOIN "ORDER" o ON d.DriverID = o.DriverID
LEFT JOIN CUSTOMER c ON o.CustomerID = c.CustomerID
WHERE o.OrderDate = TO_DATE('2024-01-01', 'YYYY-MM-DD');
```

Explanation:
This view displays a list of drivers and the clients they delivered to on a specific day. It contains the driver's name, phone number, order data, and client information (name and phone). In this example, I used **OrderDate = '2024-01-01'** to display delivery on that date.

ViewF:

```
CREATE VIEW ViewF AS
SELECT
   d.DriverID,
   d.Name AS DriverName,
   d.PhoneNumber AS DriverPhone,
```

```
    d.Suburb
FROM DRIVER d
WHERE d.DriverID NOT IN (
    SELECT DriverID
    FROM "ORDER"
    WHERE DeliveryStatus IN ('Pending', 'In Progress')
)
  AND d.Suburb = 'Mumbai Heights';
```

Explanation:

This view displays all available drivers (those not on active deliveries) in a certain suburb. It shows the driver's name, phone number, and suburb. The display excludes drivers who are currently on a delivery by verifying their **DeliveryStatus**.

ViewG:

```
CREATE VIEW ViewG AS
SELECT
    r.Name AS RestaurantName,
    COUNT(o.OrderID) AS TotalOrders
FROM RESTAURANT r
LEFT JOIN "ORDER" o ON r.RestaurantID = o.RestaurantID
GROUP BY r.Name;
```

Explanation:
This view totals the number of orders placed at each restaurant. It categorizes the data by restaurant name and returns the number of orders for each. This is useful for understanding the order volume across restaurants from the start of the business.

ViewH:

```
CREATE VIEW ViewH AS
SELECT
    r.Name AS RestaurantName,
    d.Name AS DishName,
    d.Description AS DishDescription,
    d.Price AS DishPrice,
    d.CourseType,
    d.DeliveryTimeCategory
FROM RESTAURANT r
```

```
JOIN DISH d ON r.RestaurantID = d.RestaurantID
WHERE r.RestaurantID = 1;
```

Explanation:
This view displays all of the dishes available in a particular restaurant, including their names, descriptions, prices, and categories. The view contains attributes required for creating a "booklet" for a restaurant. For this example, I've assuming **RestaurantID = 1**.

ViewI:

```
CREATE VIEW ViewI AS
SELECT
  c.Suburb,
  COUNT(o.OrderID) AS OrderCount
FROM CUSTOMER c
JOIN "ORDER" o ON c.CustomerID = o.CustomerID
WHERE o.OrderDate BETWEEN ADD_MONTHS(TRUNC(SYSDATE, 'MM'), -1) AND
LAST_DAY(ADD_MONTHS(SYSDATE, -1))
GROUP BY c.Suburb
ORDER BY OrderCount DESC;
```

Explanation:

This view determines the total number of orders per suburb over the preceding month. It classifies orders by suburb and sorts the results in descending order based on the number of orders, displaying the areas with the most order activity.

ViewJ:

```
CREATE VIEW ViewJ AS
SELECT
  c.Suburb,
  COUNT(o.OrderID) AS LateOrders,
  AVG(EXTRACT(MINUTE FROM (o.ActualDeliveryTime - o.RequestedDeliveryTime))) AS
AvgDelayMinutes
FROM CUSTOMER c
JOIN "ORDER" o ON c.CustomerID = o.CustomerID
WHERE o.OrderDate BETWEEN TO_DATE('2024-01-01', 'YYYY-MM-DD') AND
LAST_DAY(TO_DATE('2024-01-01', 'YYYY-MM-DD'))
 AND o.ActualDeliveryTime > o.RequestedDeliveryTime
GROUP BY c.Suburb;
```

Explanation:

This display shows orders that were delivered later than expected in a specific month. It displays the number of late orders by suburb and estimates the average delay in minutes. For this example, I choose **January 2024** as the specified month.

GRANT SELECT Permissions:

```
GRANT SELECT ON ViewA TO MARKERTL;
GRANT SELECT ON ViewB TO MARKERTL;
GRANT SELECT ON ViewC TO MARKERTL;
GRANT SELECT ON ViewD TO MARKERTL;
GRANT SELECT ON ViewE TO MARKERTL;
GRANT SELECT ON ViewF TO MARKERTL;
GRANT SELECT ON ViewG TO MARKERTL;
GRANT SELECT ON ViewH TO MARKERTL;
GRANT SELECT ON ViewI TO MARKERTL;
GRANT SELECT ON ViewJ TO MARKERTL;
```

Explanation:

I provided the user MARKERTL the SELECT privileges on each of the views (ViewA through ViewJ). This ensures that the user has read-only access to the views. By utilizing SELECT, I enabled MARKERTL to query the views without changing the underlying data. The SELECT privilege allows the marker to validate the content of the views and test database functionality while restricting the ability to change the data. This approach assures data integrity and avoids unauthorized changes during the grading process.

Sample Data Description:

For the sample data used in this assignment, I added entries to the ORDER table to ensure the database is ready for testing views and transactions. This is the same data that I used to populate the tables in Question 3, ensuring continuity throughout the implementation. To keep the data updated and relevant to current dates, I used SYSTIMESTAMP with intervals (such as + INTERVAL '1' HOUR or - INTERVAL '1' DAY). This strategy ensures that the sample data remains accurate without relying on a fixed date, as it will automatically update to the current date and time. By doing so, I guaranteed that all testing and validation processes were tightly aligned with real-time events, making my data more dependable and adaptive.

For Question 4, after generating all the views, I used the COMMIT command to save the changes to the database. In addition, I sent another COMMIT command after providing the MARKERTL user the requisite view permissions. I used the COMMIT command four times: twice

in Question 3 (after creating tables and granting permissions) and twice in Question 4 (after building views and granting permissions).